Noctyra / PICCP: A Post-Quantum Secure Messaging System with Pairwise Identity Continuity

Luiz Widmer - Independent Researcher

Version 0.8 - June 2026

Abstract

PICCP (Pairwise Identity Continuity Communication Protocol) is a post-quantum secure messaging protocol centered on selective identity continuity rather than permanent global identity. Noctyra is the reference client and Noctyra Relay is the reference relay implementation. Together they use pure post-quantum identity and session establishment with ML-DSA-65 and ML-KEM-768, symmetric ratcheting for forward secrecy, selective identity rotation and identity burn, encrypted attachment transfer, relay-backed group coordination, coordinator-assisted federation, and pull-only message delivery without centralized push infrastructure.

PICCP is intentionally pragmatic. It provides end-to-end confidentiality and pairwise continuity while remaining explicit about unresolved network-anonymity problems. The protocol minimizes metadata rather than claiming to eliminate it, and it is structured so stronger anonymity layers, such as mixnet or PIR-assisted retrieval, can be added without discarding the identity, relay, and continuity model.

1. Introduction

1.1 Motivation

Most deployed messaging systems still inherit one or more structural privacy weaknesses:

- global identifiers such as phone numbers, usernames, or long-lived public handles
- classical public-key cryptography vulnerable to future quantum attacks
- central key-distribution or notification infrastructure
- coarse identity semantics where a user is expected to remain the same entity forever

PICCP was designed against that backdrop. Its central claim is that identity continuity is not the same thing as permanent identity. A user may need to prove continuity to a chosen contact while being intentionally unlinkable to everyone else. This leads to a model where identity is not a public social anchor but a cryptographic state that can be rotated, archived, or burned.

1.2 Design stance

PICCP prioritizes deployable post-quantum confidentiality and selective continuity over idealized anonymity claims. The protocol therefore adopts relays, bounded metadata minimization, and coordinator-assisted federation while leaving PIR, mixnet transport, and MLS-class groups as future extensions rather than pretending they are already solved.

2. System Model

2.1 Components

The system has three primary components:

- the Noctyra client for iOS and macOS
- the Noctyra Relay for macOS
- a Linux relay deployment path with Docker parity for relay protocol behavior

The client manages identities, contacts, conversations, groups, attachments, and local security controls. The relay stores encrypted envelopes and encrypted attachment chunks, exposes transport endpoints, and optionally participates in curated or open federations.

2.2 Identity and addressing

Each active identity owns:

- an ML-DSA signing keypair
- an ML-KEM agreement keypair
- an inbox routing address
- a home relay selection
- prekey state for session bootstrap

Inbox routing addresses are bech32-encoded capability-style addresses. They are used for relay routing and mailbox lookup. They are not meant to replace the larger contact-sharing payload used for trust establishment. In practice, the human-shareable pairing object still contains the substantial cryptographic material needed for a contact relationship.

2.3 Pairwise continuity

PICCP treats continuity as pairwise, not universal. Contacts may learn that a new keyset belongs to the same peer only if that peer explicitly discloses continuity to them. If a user burns an identity, continuity can be selectively withheld. This is not a side feature. It is one of the main privacy properties of the design.

3. Threat Model

3.1 Adversaries considered

PICCP is designed against the following practical adversaries:

- passive network observers
- active relay operators
- relays that attempt metadata collection, replay, mailbox draining, or message loss
- future quantum-capable adversaries recording traffic today for later decryption
- temporary endpoint compromise
- local OS services that may expose screenshots, screen recording, clipboard, notifications, camera, or microphone handling paths

3.2 Security goals

PICCP targets:

- post-quantum resistance for long-term identity and session establishment
- end-to-end confidentiality for message and attachment payloads
- forward secrecy and bounded post-compromise recovery through ratcheting
- pairwise authenticity and signed continuity operations
- metadata reduction at the relay layer through capability-style inbox routing and temporal bucketing
- explicit user control over identity rotation, identity burn, relay choice, and local-device protections
- authenticated relay access, explicit delivery acknowledgement, and actor-proof mutation controls for operations that change shared relay state

3.3 Non-goals and limits

PICCP does not claim:

- strong global anonymity against a nation-state-grade traffic analyst
- protection against a fully compromised operating system
- protection against a malicious device vendor or kernel
- PIR-grade hidden retrieval
- mixnet-grade timing resistance
- MLS-equivalent formal group security proofs
- guaranteed closed-app delivery without a client polling window

PICCP is therefore best understood as a post-quantum, continuity-aware encrypted messenger with metadata minimization, not as a finished anonymous network.

4. Cryptographic Construction

4.1 Public-key primitives

The implementation uses the mature `liboqs` stack and instantiates:

- ML-DSA-65 for signatures and continuity assertions
- ML-KEM-768 for prekey bundles, session bootstrap, and root-ratchet refresh

The choice is intentionally conservative. Identity continuity is long-lived and must remain credible against archive-now, decrypt-later attacks. Session bootstrap must resist the same archive threat.

4.2 Symmetric primitives and key derivation

Message payloads and attachment chunks are encrypted with AES-256-GCM. Key derivation uses HKDF-SHA256 and HMAC-SHA256.

The system follows the same architectural split:

- post-quantum public-key operations to establish or refresh shared secrets
- symmetric-key ratcheting to amortize encryption work and provide forward secrecy

4.3 Prekey bundle flow

PICCP uses a post-quantum prekey-bundle flow analogous in role to PQ-X3DH:

- each identity publishes a signed prekey and individually identity-signed one-time prekeys to its relay
- the initiator fetches a bundle from the relay
- ML-KEM encapsulations derive the bootstrap shared secret
- the resulting material seeds the session root state and chain state

Prekey upload, fetch, one-time consumption, and bootstrap validation are part of the protocol implementation.

4.4 Ratchet design

After bootstrap, the system ratchets with:

- per-message symmetric chain advancement
- replay and out-of-order hardening
- periodic ML-KEM root-ratchet refresh for post-compromise recovery

In operational terms, this means:

- old message keys are not retained indefinitely
- stale and replayed counters are rejected
- session desynchronization can be healed without treating every mismatch as fatal

The client includes silent mismatch recovery paths rather than surfacing every ratchet disturbance directly to the user.

5. Pairing, Trust, and Identity Lifecycle

5.1 Pairing and trust bootstrap

Pairing is explicit. A contact relationship is created from a contact-share payload containing the cryptographic material needed to form a trust relationship, not merely from a short inbox address. The supported pairing paths are:

- QR transfer, including animated QR frames for large payloads
- password-protected contact-share files suitable for AirDrop or file transfer
- relay-mediated pairing requests with explicit metadata-leakage warnings

The relay-mediated path is intentionally described as metadata-leaky rather than plaintext-insecure. It can simplify onboarding, but it exposes more timing and discovery metadata to the relay than an offline QR or file exchange.

5.2 Identity creation

Onboarding creates an identity explicitly. The user chooses relay configuration, privacy acceptance, storage protection mode, and app-lock posture during first-run setup.

5.3 Identity rotation

Identity rotation preserves inbox continuity while replacing signing and agreement keys. A signed rotation statement links the new keyset to the prior one. Chosen contacts can verify the continuity event and continue messaging without creating an unrelated new trust relationship.

Rotation is therefore appropriate when the user wants to remain the same person to selected contacts while refreshing cryptographic state.

5.4 Identity burn

Identity burn is materially different. Burn is a severance operation. The client can selectively notify only contacts marked in advance as eligible to receive the successor identity. Everyone else loses continuity and cannot continue interaction under the burned identity's recipient material.

This distinction is central to the system:

- rotation means "same relationship, new keys"
- burn means "new entity unless I explicitly carry you forward"

5.5 Identity book and audit

The system includes multiple active or archived identities per client, each with its own home relay. Continuity-relevant actions are recorded in a continuity audit trail that can be reviewed or purged.

6. Relay Architecture

6.1 Relay role

Relays are not trusted for plaintext. They store:

- encrypted message envelopes
- encrypted attachment chunks
- prekey bundles
- relay-backed group registry state
- federation directory and coordinator state where applicable

The relay sees routing metadata, timing, protocol operation types, and policy-relevant fields, but not plaintext message or attachment contents.

Relay fetch and state-mutation operations are not unauthenticated mailbox reads. Inbox-access keys and actor proofs bind sensitive operations to identity-held signing material, and explicit acknowledgement allows clients to remove delivered messages from relay storage without relying on crash-prone implicit deletion.

6.2 Storage

Relay state persists through a normalized SQLite-backed store. The relay writes structured domain tables for inbox registrations, envelopes, attachments, prekey bundles, federation nodes, coordinator pins, groups, and join requests. Corrupt persisted rows are skipped at row scope where possible instead of reviving obsolete snapshot formats. This provides durability, structured persistence, and avoids the fragility of large flat-file state.

Attachment storage is bounded by:

- chunk size limits
- chunk count limits
- relay-configurable default and maximum TTL

This keeps the relay from silently becoming an unbounded object store.

6.3 Transport support

The relay supports multiple transport modes:

- raw TCP
- HTTP
- WebSocket

TLS can be handled in two ways:

- relay-managed TLS with a local PKCS#12 identity
- reverse-proxy TLS where HTTPS or WSS is terminated upstream and the relay stays internal

This is important operationally because many deployments are simpler behind a normal reverse proxy than through direct TCP exposure.

6.4 Relay policy

Relay policy controls include:

- relay password protection
- group-creation allow or deny mode
- temporal bucketing schedule
- attachment retention policy
- federation mode and coordinator configuration
- text-only mode for operators who do not want to host attachment chunks

Temporal bucketing can be single-bucket or multi-bucket. The multi-bucket path intentionally adds timing ambiguity to reduce the ease of correlating users by strict fetch cadence.

Relays may also advertise optional hidden-retrieval cover-query support. In that mode, compatible clients can request fixed-size cover sets from temporal buckets and extract the target record locally. This is a deployable metadata-reduction feature and an implementation stepping stone toward stronger PIR, but it is not full cryptographic PIR.

6.5 Decentralized wake and pull delivery

The architecture is fully decentralized in the delivery path. The system does not rely on APNs or any equivalent centralized push-notification provider. Closed-app instant wake is excluded because it would introduce a credential-holding notification authority inconsistent with the decentralization model.

The system therefore uses relay polling and foreground/background client fetch behavior rather than centralized push. Relays may advertise a decentralized wake policy for compatible clients: pull-only polling bounds, deterministic jitter, failure backoff, and bounded long-poll timeout support. This improves active or background fetch behavior without creating a central notification service. It does not claim guaranteed closed-app delivery on operating systems that suspend the app.

7. Federation

7.1 Modes

The relay supports three federation modes:

- solo
- curated
- open

These modes are not cosmetic labels. They affect routing rules and compatibility expectations.

7.2 Curated federation

Curated federation uses:

- allow-listed peers
- coordinator-assisted directory information
- optional signed directory snapshots
- quorum policy for forwarding decisions

This creates a managed universe where forwarding can be restricted to approved relays and where directory responses can be authenticated.

7.3 Open federation

Open federation operates in a coordinator-assisted form with optional signed-record discovery experiments. Nodes register, advertise health, and exchange directory information through coordinator infrastructure. Reachability checks, throttling, public-endpoint restrictions, signed directory validation, and freshness filtering are part of the design. A production-grade autonomous public-network adapter such as BEP5 or libp2p remains out of release scope.

In other words, open federation is implemented for coordinator snapshots, bounded peer exchange, and HTTP sidecar or native overlay experiments, but it is not an unbounded autonomous public DHT network in the release profile.

7.4 Relay-to-relay forwarding hardening

Relay-to-relay forwarding includes the following hardening properties:

- forwarding timeouts prevent stalled peer exhaustion
- actor-proof replay is rejected via nonce replay caches
- curated forwarding isolates client auth from relay-to-relay auth
- Linux relay parity was brought into line with macOS relay behavior

The Linux relay path is part of the supported deployment model rather than a transport shim.

8. Groups and Attachments

8.1 Groups

PICCP supports groups through relay-backed coordination while the group cryptography path is MLS-derived. Current group state is controlled through actor proofs and signed group commits for title edits, member additions, member removals, and self-leave operations. Each signed commit is bound to the group ID, actor fingerprint, base epoch, and previous transcript hash so stale or replayed membership edits are rejected. Group descriptors carry a required MLS epoch state containing a tree hash, confirmed transcript hash, ciphersuite label, and last commit summary. Approved joins also advance the epoch with a `joinApprove` commit summary. The relay coordinates membership and registry state but does not receive plaintext group messages. Supported flows include:

- create
- list
- update
- join request
- approval
- rejection
- leave
- creator-side delete or extinguish

This design is compatible with the relay architecture and provides practical group coordination, but it should not be misrepresented as a complete MLS deployment or a formally proven group ratchet yet. Relays advertise their group security model so clients can distinguish pairwise-fan-out groups from `mlsDerivedTree` groups. The implementation direction is to keep relay registry coordination while moving join approvals, message authentication data, and message keys into an MLS-derived tree and transcript model.

8.2 Attachments

Attachments are end-to-end encrypted, chunked, and relay-stored under TTL policy. The client applies additional controls:

- image compression and dimension bounding before upload
- attachment quotas to limit abuse and storage blow-up

- secure camera capture option for users who prefer an in-app capture path
- voice-message capture and encrypted transfer
- text-only relay mode when operators choose not to store attachment chunks

The attachment path is deliberately constrained to preserve relay boundedness and endpoint control.

9. Client Security and Local Protections

9.1 Storage protection

The client offers storage-protection modes that distinguish between Keychain-backed protection and device-only protection. Local state and attachments remain encrypted at rest. Attachment bytes are moved through scoped encrypted-to-decrypted use windows, and sensitive temporary handling is designed to minimize long-lived plaintext in application state.

9.2 App lock and coercion-oriented controls

The client includes:

- biometrics-only, PIN-only, and biometrics-plus-PIN unlock modes where supported
- session timeout controls
- reauthentication for sensitive settings changes
- action pins capable of triggering destructive or sanitizing flows

Action PIN plans can combine operations such as app reset, identity burn, identity deletion, group deletion, chat/contact deletion, photo/document wiping, storage corruption, and decoy-state creation. After use, an action PIN is consumed and promoted to the unlock PIN so it does not remain as a separate reusable trigger. These flows are explicitly defensive and are part of the operational-security posture of the app rather than mere cosmetic security settings.

9.3 Screen and capture protections

The Apple clients include UI-level protections against screenshots, screen recording, and external display exposure on supported surfaces. Sensitive panes can be redacted behind secure containers and reveal gates. These mechanisms reduce casual capture and improve user awareness, but they are not equivalent to defeating a hostile OS.

9.4 Secure typing and local input

Secure typing is user-selectable. Users can choose Apple's native secure text entry path, which preserves system behavior but may show system password affordances, or Noctyra's app-owned secure keyboard, which avoids the Apple password shortcut path. The app-owned keyboard includes letter, number, symbol, emoji, press-preview, delete-repeat, and long-press alternate-character layouts while keeping message composition inside the app's input view.

10. Implementation Profile

10.1 Protocol profile

The reference implementation delivers:

- pure post-quantum identity and session establishment
- prekey bundle upload and fetch
- symmetric ratchet plus periodic ML-KEM root ratchet
- identity rotation and identity burn
- continuity event tracking and audit
- encrypted attachment and voice-message transfer
- secure camera and app-owned secure keyboard options
- relay-backed groups with actor-proof mutation controls
- solo, curated, and coordinator-assisted open federation
- TCP, HTTP, HTTPS, WebSocket, and WSS relay transports
- relay-managed TLS and reverse-proxy TLS deployment patterns
- macOS relay, Linux relay parity path, and Docker deployment support
- relay metadata advertisement for relay name, kind, transport, TLS posture, federation state, temporal bucket policy, attachment TTL, group-creation policy, operator note, and software version
- optional relay-advertised hidden-retrieval cover queries
- explicit group-security-model advertisement and required MLS epoch metadata
- relay-advertised decentralized wake policy for jittered pull or bounded long-poll clients

10.2 Deferred work

The following areas remain future work:

- full cryptographic PIR-assisted hidden retrieval
- mixnet or onion-style transport integration
- DHT-style autonomous open-federation discovery
- explicit signed join-approval commits and full MLS-derived group ratchet implementation
- external independent audit and signed release-provenance packaging
- stronger closed-app background delivery that does not require centralized push infrastructure

These are genuine open areas and remain on the roadmap because they are materially harder than the deployed protocol profile.

11. Conclusion

PICCP is an implemented post-quantum messaging system with selective identity continuity, relay-backed deployment, and a clear separation between delivered security properties and deferred anonymity work. The Noctyra implementation is not a finished anonymity network, but it is a working encrypted messenger that enforces the core ideas that motivate the protocol:

- identity need not be permanent

- long-term continuity should survive quantum-era attacks
- relays should not hold plaintext trust
- networking should remain deployable without surrendering future upgrade paths

Further work focuses on hardening, stronger metadata protection, and future anonymity upgrades while preserving the same design direction.